

## Question 1

Run from Assignment1/Exercise1/week01exercises/ the command `gradle run -PmainClass=exercises01.CounterThreads2Covid`

```
public class CounterThreads2Covid {

    long counter = 0;
    final long PEOPLE = 10_000;
    final long MAX_PEOPLE_COVID = 15_000;
    Lock l = new ReentrantLock();

    public CounterThreads2Covid() {
        try {
            Turnstile turnstile1 = new Turnstile();
            Turnstile turnstile2 = new Turnstile();

            turnstile1.start();turnstile2.start(); // start two threads
            turnstile1.join();turnstile2.join(); // join two threads

            System.out.println(counter+" people entered"); // check threads stopped at correct
        }
        catch (InterruptedException e) {
            System.out.println("Error " + e.getMessage());
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        new CounterThreads2Covid();
    }

    public class Turnstile extends Thread {

        public void run() {
            for (int i = 0; i < PEOPLE; i++) {
                l.lock(); // critical section between lock/unlock; evaluating / updating shared
                try {
                    if (counter >= MAX_PEOPLE_COVID) {
                        return;
                    }
                    counter++;
                } finally { // use try-finally to always unlock, avoiding deadlocks
                    l.unlock();
                }
            }
        }
    }
}
```

## Question 2

Run from Assignment1/Exercise2/week02exercises/ the command `gradle run -PmainClass=exercises02.ReadersWriters`

```
public class FairReadWriteMonitor {
    private int readers      = 0;
    private boolean writer   = false;

    public synchronized void readLock() {
        try {
            while(writer)
                this.wait();
            readers++;
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public synchronized void readUnlock() {
        readers--;
        if(readers==0)
            this.notifyAll();
    }

    public synchronized void writeLock() {
        try {
            while(writer)
                this.wait();
            writer=true;
            while(readers > 0)
                this.wait();
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public synchronized void writeUnlock() {
        writer=false;
        this.notifyAll();
    }
}
```

### Question 3

Run from Assignment1/Exercise2/week02exercises/ the command gradle run -PmainClass=exercises02.TestMutableInteger

```
public class TestMutableInteger {
    public static void main(String[] args) {
        final MutableInteger mi = new MutableInteger();
        Thread t = new Thread(() -> {
            while (mi.get() == 0)           // Loop while zero
                /* Do nothing*/ }
            System.out.println("I completed, mi = " + mi.get());
        });
        t.start();
        try { Thread.sleep(500); } catch (InterruptedException e) { e.printStackTrace(); }
        mi.set(42);
        System.out.println("mi set to 42, waiting for thread ...");
        try { t.join(); } catch (InterruptedException e) { e.printStackTrace(); }
        System.out.println("Thread t completed, and so does main");
    }
}

class MutableInteger {
    private volatile int value = 0; // made volatile to make sure it is flushed when updated
    public void set(int value) { // might have synchronized these two methods instead
        this.value = value;
    }
    public int get() {
        return value;
    }
}
```

### Question 4

Run from Assignment2/Exercise3/week03exercises/ the command gradle run -PmainClass=exercises03.CountingThreads

```
public class CountingThreads {
    int count;
    ReentrantLock l;

    public CountingThreads() throws InterruptedException {
        count = 0; // (init(c)) - variable access
        l = new ReentrantLock(); // (init(l)) - variable access

        CountingThread t1 = new CountingThread(); // (init(t1))
        CountingThread t2 = new CountingThread(); // (init(t2))
    }
}
```

```

        t1.start(); // (start(t1)) - synchronization
        t2.start(); // (start(t2)) - synchronization

        t1.join(); // (join(t1)) - synchronization
        t2.join(); // (join(t2)) - synchronization

        System.out.println("count="+count); // (3) - variable access
    }

    public class CountingThread extends Thread {
        public void run() {
            l.lock(); // (4) - synchronization
            int temp = count; // (1) - variable access
            count = temp + 1; // (2) - variable access
            l.unlock(); // (5) - synchronization
        }
    }

    public static void main(String[] args) throws InterruptedException {
        new CountingThreads();
    }
}

```

### Happens-Before Order

$HB_{po}^m = m(\text{init}(c)) \rightarrow m(\text{init}(l)) \rightarrow m(\text{init}(t1)) \rightarrow m(\text{init}(t2)) \rightarrow m(\text{start}(t1))$   
 $\rightarrow m(\text{start}(t2)) \rightarrow m(\text{join}(t1)) \rightarrow m(\text{join}(t2)) \rightarrow m(3)$

$HB_{po}^{t1} = t1(4) \rightarrow t1(1) \rightarrow t1(2) \rightarrow t1(5)$

$HB_{po}^{t2} = t2(4) \rightarrow t2(1) \rightarrow t2(2) \rightarrow t2(5)$

$HB_{init} = \{m(\text{start}(t1)) \rightarrow t1(4), m(\text{start}(t2)) \rightarrow t2(4)\}$

$HB_{ter} = \{t1(5) \rightarrow m(\text{join}(t1)), t2(5) \rightarrow m(\text{join}(t2))\}$

### Synchronization Orders

$m(\text{start}(t1)), m(\text{start}(t2)), t1(4), t1(5), t2(4), t2(5), m(\text{join}(t1)), m(\text{join}(t2))$   
 $m(\text{start}(t1)), t1(4), m(\text{start}(t2)), t1(5), t2(4), t2(5), m(\text{join}(t1)), m(\text{join}(t2))$   
 $m(\text{start}(t1)), t1(4), t1(5), m(\text{start}(t2)), t2(4), t2(5), m(\text{join}(t1)), m(\text{join}(t2))$   
 $m(\text{start}(t1)), m(\text{start}(t2)), t2(4), t2(5), t1(4), t1(5), m(\text{join}(t1)), m(\text{join}(t2))$

## Question 5

Run from Assignment2/Exercise4/week04exercises/ the command `gradle run -PmainClass=exercises04.PersonTester`

```
public class Person {                                     // all vari
    private static long currentId;                         // internal
    private static boolean firstPersonInitialized = false; // internal
    private final long id;                                 // immutabl
    private final String name;                             // immutabl
    private int zip;                                        // init def
    private String address;                                // init def

    public Person() {                                     // default c
        this(0);
    }

    public Person(long startId) {                           // construc
        synchronized (Person.class) {
            if (!firstPersonInitialized) {
                currentId = startId;
                firstPersonInitialized = true;
            }
            id = currentId;
            name = "John Doe";
            currentId++;
        }
    }

    public synchronized void changeAddress(String address, int zip) { // mutable j
        this.zip = zip;
        this.address = address;
    }

    public long getId() {                                    // immutabl
        return id;
    }

    public String getName() {                                // immutabl
        return name;
    }

    public synchronized int getZip() {                       // mutable j
        return zip;
    }
}
```

```

    public synchronized String getAddress() {                                // mutable
        return address;
    }
}

```

## Question 6

Run from Assignment3/Exercise5/week05exercises/ the command `gradle cleanTest test --tests exercises05.ConcurrentSetTest`

```

public class ConcurrentSetTest {

    // Variable with set under test
    private ConcurrentIntegerSet set;

    @BeforeEach
    public void initialize() {
        // set = new ConcurrentIntegerSetBuggy(); // for 5.1.1, 5.1.2
        set = new ConcurrentIntegerSetLibrary(); // for 5.1.4
    }

    // @Disabled
    @RepeatedTest(10000)
    @DisplayName("Adding Single element to Set")
    public void addSingle() throws InterruptedException {
        int nrThreads = 32;
        Random r = new Random();
        int e = r.nextInt();

        AtomicInteger trueCount = new AtomicInteger();

        CyclicBarrier barrier = new CyclicBarrier(nrThreads+1);

        for (int i = 0; i < nrThreads; i++) {
            new Thread(() -> {
                try {
                    barrier.await();
                    boolean s = set.add(e);
                    trueCount.getAndAdd(s ? 1 : 0);
                    barrier.await();
                } catch (InterruptedException | BrokenBarrierException exc) {
                    exc.printStackTrace();
                }
            }).start();
        }
    }
}

```

```

        try {
            barrier.await();
            barrier.await();
        } catch (InterruptedException | BrokenBarrierException exc) {
            exc.printStackTrace();
        }

        assertEquals(set.size(), 1);
        assertEquals(trueCount.get(), 1);
    }

    @RepeatedTest(10000)
    @DisplayName("Remove Single element from Set")
    public void removeSingle() throws InterruptedException {
        int nrThreads = 32;
        Random r = new Random();
        int e = r.nextInt();
        set.add(e);

        AtomicInteger trueCount = new AtomicInteger();

        CyclicBarrier barrier = new CyclicBarrier(nrThreads+1);

        for (int i = 0; i < nrThreads; i++) {
            new Thread(() -> {
                try {
                    barrier.await();
                    boolean s = set.remove(e);
                    trueCount.getAndAdd(s ? 1 : 0);
                    barrier.await();
                } catch (InterruptedException | BrokenBarrierException exc) {
                    exc.printStackTrace();
                }
            }).start();
        }

        try {
            barrier.await();
            barrier.await();
        } catch (InterruptedException | BrokenBarrierException exc) {
            exc.printStackTrace();
        }

        assertEquals(0, set.size());
        assertEquals(1, trueCount.get());
    }
}

```

## Question 7

Run from Assignment4/Exercise9/week09exercises/ the command `gradle run -PmainClass=exercises09.TestCountPrimesThreads`

```
public class TestCountPrimesThreads {

    public static void main(String[] args) { new TestCountPrimesThreads(); }

    public TestCountPrimesThreads() {
        Benchmark.SystemInfo();
        final int range= 100_000;
        Benchmark.Mark7("countSequential", i -> countSequential(range));
        for (int c= 1; c<=32; c++) {
            final int threadCount = c;
            Benchmark.Mark7(String.format("countParallelN %7d", threadCount),
                i -> countParallelN(range, threadCount));
        }
    }

    private static boolean isPrime(int n) {
        int k = 2;
        while (k * k <= n && n % k != 0)
            k++;
        return n >= 2 && k * k > n;
    }

    // Sequential solution
    private static int countSequential(int range) {
        int count= 0;
        final int from= 0, to= range;
        for (int i= from; i<to; i++)
            if (isPrime(i)) count++;
        return count;
    }

    // General parallel solution, using multiple threads
    private static int countParallelN(int range, int threadCount) {
        final int perThread= range / threadCount;
        final PrimeCounter lc= new PrimeCounter();
        Thread[] threads= new Thread[threadCount];
        for (int t= 0; t<threadCount; t++) {
            final int from= perThread * t,
                to = (t+1==threadCount) ? range : perThread * (t+1);
            threads[t]= new Thread( () -> {
                for (int i= from; i<to; i++)
```



```

        if (isPrime(i)) lc.increment();
    });
}
for (int t= 0; t<threadCount; t++)
    threads[t].start();
try {
    for (int t=0; t<threadCount; t++)
        threads[t].join();
} catch (InterruptedException exn) { }
return lc.get();
}
}

# OS:   Mac OS X; 14.6; aarch64
# JVM:  Homebrew; 17.0.16
# CPU:   null; 8 "cores"
# Date: 2025-10-27T11:10:50+0100
countSequential          2011099,6 ns    12439,42      128
countParallelN           1      2068329,6 ns     835,60      128
countParallelN           2      1364334,1 ns     4115,08     256
countParallelN           3      1162648,4 ns     16703,89     256
countParallelN           4      1030419,1 ns     24443,32     256
countParallelN           5      1187034,4 ns     15321,28     256
countParallelN           6      1205606,4 ns     17529,10     256
countParallelN           7      1209973,6 ns     3557,38      256
...

```

## Question 8

Run from Assignment5/Exercise10/week10exercises/ the command `gradle run -PmainClass=exercises10.TestCountPrimesThreads`

```

public class TestCountPrimesThreads {
    public static void main(String[] args) {
        new TestCountPrimesThreads();
    }

    public TestCountPrimesThreads() {
        final int range = 100_000;
        Benchmark.Mark7("countSequential", i -> countSequential(range));
        for (int c = 1; c <= 32; c = 2 * c) {
            final int threadCount = c;
            Benchmark.Mark7(String.format("countParallelN %2d", threadCount),
                i -> countParallelN(range, threadCount));
            Benchmark.Mark7(String.format("countParallelNLocal %2d", threadCount),
                i -> countParallelNLocal(range, threadCount));
            Benchmark.Mark7(String.format("countParallelNExecutors %2d", threadCount),

```

```

        i -> countParallelNExecutors(range, threadCount));
    }
}

private static boolean isPrime(int n) {
    int k = 2;
    while (k * k <= n && n % k != 0)
        k++;
    return n >= 2 && k * k > n;
}

// Sequential solution
private static long countSequential(int range) {
    long count = 0;
    final int from = 0, to = range;
    for (int i = from; i < to; i++)
        if (isPrime(i))
            count++;
    return count;
}

// General parallel solution, using multiple threads
private static long countParallelN(int range, int threadCount) {
    ...
}

private static class countPrimesTask implements Callable<Integer> {
    private final int low, high;

    countPrimesTask(int l, int h) {
        low = l;
        high = h;
    }

    @Override
    public Integer call() {
        Integer count = 0;
        for (int i = low; i < high; i++) {
            if (isPrime(i))
                count++;
        }
        return count;
    }
}

private static long countParallelNExecutors(int range, int threadCount) {

```

```

final int perThread = range / threadCount;
final ArrayList<Future<Integer>> results = new ArrayList<Future<Integer>>();

ExecutorService pool = new ForkJoinPool(4);

for (int t = 0; t < threadCount; t++) {
    final int from = perThread * t,
        to = (t + 1 == threadCount) ? range : perThread * (t + 1);
    final int threadNo = t;
    Future<Integer> f = pool.submit(new countPrimesTask(from, to));
    results.add(f);
}

pool.shutdown();

long count = 0;
for (Future<Integer> j : results) {
    try {
        count += j.get().intValue();
    } catch (InterruptedException | ExecutionException e) {
    }
}

try {
    // Wait for all tasks to complete or timeout after 100 seconds
    if (pool.awaitTermination(1, TimeUnit.SECONDS)) {
        // System.out.println("All tasks completed successfully.");
    } else {
        System.out.println("Timeout elapsed before termination.");
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}

return count;
}
}

```

|                           |  |              |           |     |
|---------------------------|--|--------------|-----------|-----|
| countSequential           |  | 3786314,9 ns | 165573,12 | 128 |
| countParallelN 1          |  | 3776881,9 ns | 4031,11   | 128 |
| countParallelNExecutors 1 |  | 3819188,7 ns | 6039,99   | 128 |
| countParallelN 2          |  | 2381288,6 ns | 4940,99   | 128 |
| countParallelNExecutors 2 |  | 2473163,2 ns | 40689,79  | 128 |
| countParallelN 4          |  | 1376270,8 ns | 10340,51  | 256 |
| countParallelNExecutors 4 |  | 1422991,8 ns | 10283,99  | 256 |
| countParallelN 8          |  | 1182302,0 ns | 16004,73  | 256 |

|                            |              |          |     |
|----------------------------|--------------|----------|-----|
| countParallelNExecutors 8  | 1306301,8 ns | 14939,91 | 256 |
| countParallelN 16          | 1196126,1 ns | 23312,49 | 256 |
| countParallelNExecutors 16 | 1220702,3 ns | 9555,83  | 256 |
| countParallelN 32          | 1544783,4 ns | 12126,87 | 256 |
| countParallelNExecutors 32 | 1197267,0 ns | 71356,65 | 256 |

## Question 9

Run from Assignment3/Exercise6/week06exercises/ the command `gradle run -PmainClass=exercises06.CasHistogram`

```
class CasHistogram implements Histogram {
    private final AtomicInteger[] counts;

    public CasHistogram(int span) {
        counts = new AtomicInteger[span];
        for (int i = 0; i < span; i++) {
            counts[i] = new AtomicInteger();
        }
    }

    public int getCount(int bin) {
        return counts[bin].get();
    }

    public int getSpan() {
        return counts.length;
    }

    public void increment(int bin) {
        int val;
        do {
            val = counts[bin].get();
        } while (!counts[bin].compareAndSet(val, val + 1));
    }

    public int getAndClear(int bin) {
        int val;
        do {
            val = counts[bin].get();
        } while (!counts[bin].compareAndSet(val, 0));
        return val;
    }
}
```

## Question 10

```
class LockFreeStack<T> {
    AtomicReference<Node<T>> top = new AtomicReference<Node<T>>(); // Initializes to null

    public void push(T value) {
        Node<T> newHead = new Node<T>(value); // Pu1
        Node<T> oldHead; // Pu2
        do {
            oldHead = top.get(); // Pu3
            newHead.next = oldHead; // Pu4
        } while (!top.compareAndSet(oldHead, newHead)); // Pu5 - Linearization point for me

    }

    public T pop() {
        Node<T> newHead; // Po1
        Node<T> oldHead; // Po2
        do {
            oldHead = top.get(); // Po3 - Linearization point if stack is non-empty
            if(oldHead == null) { return null; } // Po4
            newHead = oldHead.next; // Po5
        } while (!top.compareAndSet(oldHead, newHead)); // Po6 - Linearization point for me
        // stack is non-empty

        return oldHead.value;
    }
}
```

## Question 11

Run from Assignment5/Exercise11/week11exercises/ the command `gradle run -PmainClass=exercises11.PrimeCountingPerf`

```
class PrimeCountingPerf {
    public static void main(String[] args) { new PrimeCountingPerf(); }
    static final int range= 100000;

    //Test whether n is a prime number
    private static boolean isPrime(int n) {
        int k= 2;
        while (k * k <= n && n % k != 0)
            k++;
        return n >= 2 && k * k > n;
    }

    // Sequential solution
}
```

```

private static long countSequential(int range) {
    long count = 0;
    final int from = 0, to = range;
    for (int i=from; i<to; i++)
        if (isPrime(i)) count++;
    return count;
}

// IntStream solution
private static long countIntStream(int range) {
    return IntStream.range(2, range)
        .filter(i -> isPrime(i))
        .count();
}

// Parallel Stream solution
private static long countParallel(int range) {
    return IntStream.range(2, range)
        .parallel()
        .filter(i -> isPrime(i))
        .count();
}

// parallelStream solution
private static long countparallelStream(List<Integer> list) {
    return list
        .parallelStream()
        .filter(i -> isPrime(i))
        .count();
}

public PrimeCountingPerf() {
    Benchmark.Mark7("Sequential", i -> countSequential(range));

    Benchmark.Mark7("IntStream", i -> countIntStream(range));

    Benchmark.Mark7("Parallel", i -> countParallel(range));

    List<Integer> list = new ArrayList<Integer>();
    for (int i= 2; i< range; i++){ list.add(i); }
    Benchmark.Mark7("ParallelStream", i -> countparallelStream(list));
}

```

|            |              |          |     |
|------------|--------------|----------|-----|
| Sequential | 1986799,4 ns | 11162,72 | 128 |
| IntStream  | 2011782,4 ns | 17478,71 | 128 |

|                |             |         |     |
|----------------|-------------|---------|-----|
| Parallel       | 529728,2 ns | 3814,19 | 512 |
| ParallelStream | 524621,2 ns | 5687,18 | 512 |

## Question 12

```

-module(server).
% to be extended
-export([start/2, init/2]).
-include("defs.hrl").

% 1. State

-record(server_state, {
    idle_list = [], pending_tasks = [], total_workers = 0, min_workers, max_workers
}).

% 2. Start

start(MinNumWorkers, MaxNumWorkers) ->
    spawn(?MODULE, init, [MinNumWorkers, MaxNumWorkers]).

% 3. Initialization

init(MinNumWorkers, MaxNumWorkers) ->
    State = #server_state{min_workers = MinNumWorkers, max_workers = MaxNumWorkers},
    NewState = spawn_n_workers(State#server_state.min_workers, State),
    loop(NewState).

% 4. Behavior upon receiving messages

loop(State) ->
    receive
        {work_done, WorkerPID} ->
            handle_work_done(WorkerPID, State);
        {compute, SenderPID, Tasks} ->
            handle_compute(SenderPID, Tasks, State);
        idle_workers ->
            io:format("Idle workers: ~p~n", [State#server_state.idle_list]);
        {'DOWN', _, _, _, normal} ->
            handle_normal(State);
        {'DOWN', _, _, PID, Reason} ->
            handle_error(PID, Reason, State)
    end.

% 5. Message handlers

```

```

handle_normal(State) ->
    loop(State).

handle_error(PID, Reason, State) ->
    io:format("Worker ~w crashed with error ~p~n", [PID, Reason]),
    NewState = spawn_n_workers(1, State#server_state{
        total_workers = State#server_state.total_workers - 1
    }),
    loop(NewState).

handle_compute(_, [], State) ->
    loop(State);
handle_compute(
    SenderPID,
    [Task | Remaining],
    State = #server_state{
        total_workers = TotalWorkers, max_workers = MaxWorkers, idle_list = IdleList
    }
) ->
    case IdleList of
        [] ->
            case TotalWorkers < MaxWorkers of
                false ->
                    NewState = State#server_state{
                        pending_tasks = [{Task, SenderPID} | State#server_state.pending_tasks]
                    };
                true ->
                    NewState = spawn_n_workers(1, State),
                    handle_compute(SenderPID, [Task | Remaining], NewState)
            end;
        [WorkerPID | RemainingWorkers] ->
            WorkerPID ! {compute, SenderPID, Task},
            NewState = State#server_state{idle_list = RemainingWorkers}
    end,
    handle_compute(SenderPID, Remaining, NewState).

handle_work_done(
    WorkerPID,
    State = #server_state{
        total_workers = TotalWorkers, min_workers = MinWorkers, idle_list = IdleList
    }
) ->
    case State#server_state.pending_tasks of
        [] ->
            case TotalWorkers > MinWorkers of
                false ->

```



```

        NewState = State#server_state{idle_list = [WorkerPID | IdleList]};
true ->
    WorkerPID ! stop,
    NewState = State#server_state{total_workers = TotalWorkers - 1}
end;
[Task, SenderPID] ->
    WorkerPID ! {compute, SenderPID, Task},
    NewState = State#server_state{pending_tasks = Remaining}
end,
loop(NewState).

% Private functions (to be used, e.g., by message handlers)

spawn_n_workers(0, State) ->
    State;
spawn_n_workers(NumWorkers, State) ->
    PID = worker:start(self()),
    monitor(process, PID),
    NewState = State#server_state{
        idle_list = [PID | State#server_state.idle_list],
        total_workers = State#server_state.total_workers + 1
    },
    spawn_n_workers(NumWorkers - 1, NewState).

```